OPEN ACCESS
Vietnam Journal of Computer Science
Vol. 10, No. 1 (2023) 39–54
© The Author(s)
DOI: 10.1142/S2196888822500300

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Automatic Classification of Bird Sounds: Using MFCC and Mel Spectrogram Features with Deep Learning

Silvestre Carvalho
Instituto Superior de Engenharia do Porto
Rua Dr. Bernardino de Almeida
431, 4200-072, Porto, Portugal

Elsa Ferreira Gomes
Instituto Superior de Engenharia do Porto & INESC TEC
Rua Dr. Bernardino de Almeida
431, 4200-072, Porto, Portugal
efg@isep.ipp.pt

Received 5 December 2021
Accepted 31 May 2022
Published 10 August 2022

Bird species identification is a relevant and time-consuming task for ornithologists and ecologists. With growing amounts of audio-annotated data, automatic bird classification using machine learning techniques is an important trend in the scientific community. Analyzing bird behavior and population trends helps detect other organisms in the environment and is an important problem in ecology. Bird populations react quickly to environmental changes, which make their real-time counting and tracking challenging and very useful. A reliable methodology that automatically identifies bird species from audio would therefore be a valuable tool for the experts in different scientific and applicational domains. The goal of this work is to propose a methodology to identify bird sounds. In this paper, we explore deep learning techniques that are being used in this domain, such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to classify the data. In deep learning, audio problems are commonly approached by converting them into images using audio feature extraction techniques such as Mel Spectrograms and Mel Frequency Cepstral Coefficients (MFCCs). We propose and test multiple deep learning and feature extraction combinations in order to find the most suitable approach to this problem.

Keywords: Bird species classification; deep learning; audio feature extraction.

AMS Subject Classification: 22E46, 53C35, 57S20

This is an Open Access article published by World Scientific Publishing Company. It is distributed under the terms of the Creative Commons Attribution 4.0 (CC BY) License which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

1. Introduction

With climate change, the analysis of interactions between organisms and their environment is an important problem in ecology.[1] Bird behavior and population trends enable the detection of other organisms in the environment because birds react quickly to environmental changes. It is also relevant to monitor the evolution of bird populations as recent studies have shown their widespread decline.[2] Bird identification can be done manually by experts in the area, either through audio or images,[3] but the growing amount of data make this process tedious and time consuming. It requires a lot of costly human effort.[1] The objective of this paper is to propose a methodology to identify a bird species by its chirp. We intend to explore and improve the results of deep learning techniques that are being applied in this domain.

2. State of Art

Deep learning is commonly used for sound classification in many different domains. A typical approach is to convert the audio file into an image, such a spectrogram, and use a deep neural network to process that image.[4]

2.1. *Pre-processing and feature extraction*

Regarding the audio processing, it is common to normalize the sound signal and use filters for noise removal. In particular, the syllable segmentation algorithm has been used in frog sounds[5,6] and in bird sounds.[7]

The appropriate signal features extraction is crucial for the process of bird species recognition, as the acoustical environment of bird audio recordings contain noise in the signal, and there is a big diversity among bird species songs.[8] A Mel Spectrogram is a spectrogram where the frequencies are converted to the mel scale. This is a perceptual scale of pitches such that equal distances in pitch sound equally distant to the listener.[9] The Mel-Frequency Cepstral Coefficients (MFCC) are widely used in audio classification problems, due to their computational efficiency and noise robustness.[10] Kortas[11] used MFCC for sound-based bird classification and stated that the idea behind the mel scale is that it relates to the way humans hear.

2.2. *Deep learning for sound classification*

Deep Learning is a subfield of machine learning that allows computational models composed of multiple processing layers to learn representations of data with multiple levels of abstraction. Deep learning methods have a very good ability to process audio.[12]

A Convolutional Neural Network (CNN) is a deep learning architecture, which is able to learn image classification from raw image data, using representation learning. Its architecture is analogous to the connectivity pattern of the neurons in the human brain, being inspired by the organization of the visual cortex.[13] A Recurrent Neural Network (RNN) is designed to recognize patterns in sequential data. RNNs have feedback loops to keep the context of previously seen data and take those in

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

consideration for classification or prediction. The algorithms used in RNN take time and sequence into account, meaning they have a temporal dimension. RNNs are applicable to images since these can be decomposed into a series of patches and treated as a sequence.[14] Long Short-Term Memory (LSTM) networks are a modified version of recurrent neural networks. These networks were created with the intent of fixing computational issues of the original RNN.[15] Regular RNNs have a vanishing gradient problem that may quickly lead the learning process to a halt. LSTM was designed with the intent of overcoming these error back-flow problems.[16] LSTM is capable of learning long-term dependencies. Retaining long-term temporal features in audio classification may help to improve the accuracy of the bird classification task given that context is kept. LSTM can remember information for long periods of time as their default behavior as they are explicitly designed to avoid the long-term dependency problem.[17] Gated Recurrent Unit (GRU) networks are another improvement of standard RNN. GRU can be considered as a variant of LSTM since both are designed similarly and, in some cases, produce similar results.[18] Convolutional Recurrent Neural Network (CRNN) is a hybrid between CNN and RNN. A CRNN is a modified CNN with the last convolutional layers replaced with an RNN. In CRNN, CNN and RNN play the roles of feature extractor and temporal summarizer, respectively. Adopting an RNN for feature aggregation enables the networks to take the global structure into account while local features are extracted by the remaining convolutional layers.[19]

2.3. *Other approaches*

Lasseck[20] achieved top scores in the 2019 challenge BirdCLEF. The goal of the challenge was to localize and identify all audible birds within the provided soundscape test set. Each soundscape was divided into segments of 5 s.[21] The used evaluation metric was the Classification Mean Average Precision (cmAP). For each class 'c', all predictions classified as that class are obtained and ordered by decreasing probability to compute the average precision for that class. The mean for all classes is the main evaluation metric.

Hiatt[22] compiled the dataset used in this work and proposed an approach. This dataset contains a subset of records, labeled by species from California and Nevada.

A pre-processing methodology was used where a data generator combines Mel Spectrograms with MFCC into a single two-dimensional array. The deep learning model implemented is a CNN with three stacks of 2D Convolutions using ReLU activation and MaxPooling layers followed by dropout layers with a rate of 0.2. The model architecture is topped by a global average pooling layer, which is followed by a fully connected dense layer using the softmax activation. The accuracy metric is used to evaluate the performance of the models on a 33% holdout methodology. The documented results correspond to the model from the epoch with the minimum validation loss value. It achieved a validation accuracy of 19.27% and a training accuracy of 20.44%.[22]

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

3. Our Approach

The dataset used in this work contains a set of recordings, labeled by species, from California and Nevada, USA.[22] It contains 91 species, with 30 sound sample files per species. In total there are 2,730 samples (MP3 file format), ranging from less than 1 s to 195 s. The sum of the duration of all samples from all the species is 20 h, 25 min, and 8 s (73508 s). The proposed methodology has three main steps:

- Pre-processing,
- Feature extraction,
- Deep learning modeling.

In Fig. 1, we present the methodology, and it is described in the following sections.

3.1. *Pre-processing*

The audio files were converted from MP3 to WAV file format. Since the original files had sample rates between 22.5 kHz and 44.1 kHz, we normalized all the files to the minimum sample rate of 22.5 kHz (Fig. 2). The file size difference is a downside, because while the dataset using the MP3 file format takes 1.4 GB, the WAV file format version takes 9 GB.

Before the feature extraction, the audio data can be further improved using multi-threading support to speed up the pre-processing. This process starts with converting the audio channels from Stereo to Mono. Then, an envelope filter can be applied to the audio data. This envelope basically removes parts of the audio that are below a certain threshold and can be considered background noise. Figure 3 presents the padding on a signal (of the species Black-tailed Gnatcatcher) to be used on a split

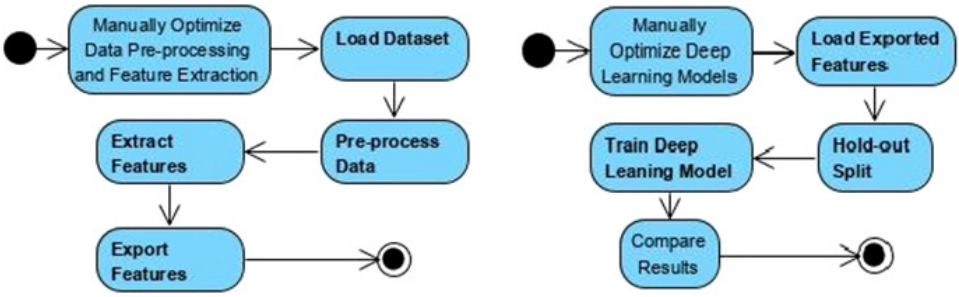

Fig. 1. Deep learning model.

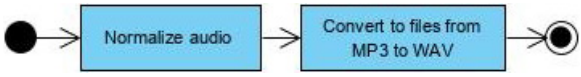

Fig. 2. Pre-processing.

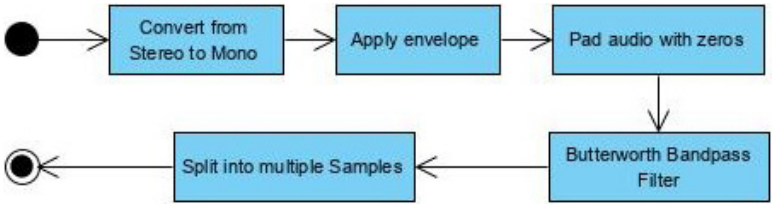

Fig. 3. Removing noise and splitting multiple samples.

with a window length of 3 s. Half of the window length is added to the beginning and the end of the signal (1.5 s).

The Butterworth bandpass filter[22] is used to remove frequencies outside of the bird vocalization range. The value used for the lowest frequency cut is 1500 Hz and that for the highest is 8000 Hz. While testing the pre-processing of the audio data, the Butterworth bandpass filter attenuates most of the noise, such as rain or wind, which in some cases makes a big difference both visually and audibly regarding the clarity of the bird chirps.

In Fig. 4, we can also see the result of the Butterworth bandpass filter applied to the signal. The final step is splitting the audio data into multiple samples. This process begins by finding the peaks in the sound. These peaks are obtained using a function from the Python library SciPy. This function has user definable parameters such as peak threshold and minimum distance between peaks. After getting a list of peaks, the audio data are then split into multiple samples, centered at the peak, and all with the same length of 3 s.

3.2. *Feature extraction*

We used MFCC to extract features from the signal, using Python library python_speech_features[23] (Fig. 5). The parameters of this function are as follows: Sample_rate; numcep, the number of cepstrum (default is 13); nfilt, the number of filters in the filterbank (default is 26); nfft, the FFT size (default is 512); and the

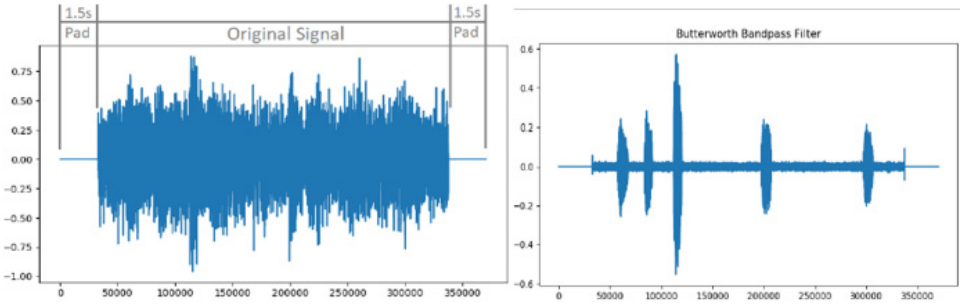

Fig. 4. Signal and Butterworth bandpass filter applied to the sign.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

```
from python_speech_features import mfcc

def get_mfccs(time_series, sample_rate):
    nfft = (round(sample_rate / 40))
    return mfcc(time_series, sample_rate, numcep=13, nfilt=26, nfft=nfft
    , lowfreq=1500, highfreq=8000).T
```

Fig. 5. MFCC extraction code.

```
from librosa.feature import melspectrogram

def get_melspectrogram(time_series, sample_rate):
    return melspectrogram(y=time_series, sr=sample_rate, n_fft=1024,
    hop_length=1024, n_mels=128, htk=True, fmin=1500, fmax=8000)
```

Fig. 6. Mel spectrogram extraction code.

other two correspond to lowest band edge of mel filters (in Hz, default is 0), and highest band edge of mel filters (in Hz, default is sample rate/2).

Mel spectrograms were extracted using the Python library LibROSA[24] (Fig. 6). The parameters of this function are sample_rate; n_fft, length of the FFT window (default is 2048); hop_length, number of samples between successive frames (default is 512); n_mels, number of Mel bands to generate (default is 128), and the lowest and highest frequency (in Hz).

In Fig. 7, we present a visualization of the extracted features from the audio segment shown in Fig. 1. The obtained images are then saved into JPEG files. The Python library Matplotlib[25] was used to obtain the images.

3.3. *Deep learning model architectures*

In this section, we describe the deep learning models we have used. These models are implemented using the TensorFlow framework[26] with the Keras back-end[27] for Python. These models are the result of analyzing and testing multiple existing deep

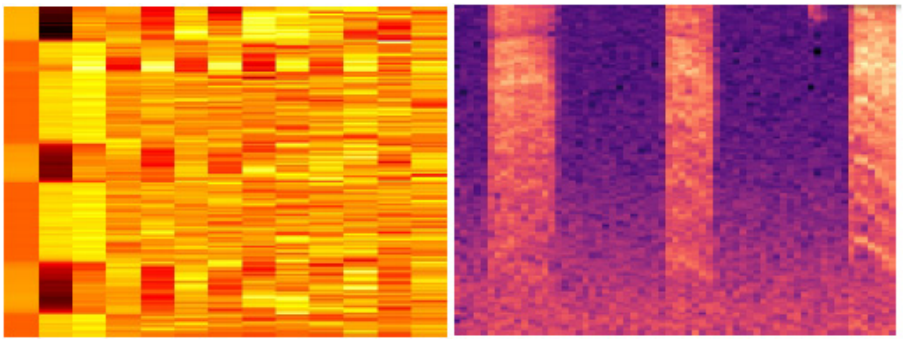

Fig. 7. Extracted MFCC (left) and mel spectrogram (right).

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 1. Convolutional neural network — Model architecture.

	Type	Kernel	Notes	Input shape
1	Reshape		TimeDistributed, Target = −1	$224 \times 224 \times 3$
2	Dense	64	TimeDistributed; Activation = tanh	224×672
3	GRU	128	BiDirectional	224×64
4	Dense	64	Activation = relu	224×256
5	MaxPoolinglD			224×64
6	Dense	32	Activation = relu	112×64
7	Flatten			112×32
8	Dropout		Rate = 0.5	3584
9	Dense	32	Activation = relu	3584
10	Dense	91	Activation = softmax	32

learning model architectures available online, such as the implementations by Adams.[28] The models were adapted and optimized to our problem, changing the network architecture (adding layers and changing dimensions) and hyperparameters. For instance, the CNN architecture, presented in the next section, adds batch normalizations layers to make results more stable and uses the activation function tanh instead of relu in the first Conv2D layer. The dropout rate was increased to 0.5 to reduce overfitting. Since we are dealing with a larger dataset with more classes, we also adopted an extra dropout layer. Despite also being a sound classification problem, our dataset contains sounds of a very different nature (bird sounds *vs* musical instruments).

3.3.1. *Convolutional neural network*

The selected CNN architecture is composed of six convolutional layers (Conv2D), each followed by a pooling layer (MaxPooling2D). As we can see in Table 1, between each convolutional and pooling layer pair, a batch normalization layer is added to reduce the amount of shift on the values of the hidden layers and increase the learning speed and reducing overfitting.[29] After a flattened layer, a dropout layer with a rate of 50% is added to reduce overfitting. The last layer is the fully connected layer, which is a dense layer with the softmax activation. In total, this model has 9,33,211 trainable parameters.

4. Recurrent Neural Network — Long Short-Term Memory

The chosen Long Short-Term Memory (LSTM) model architecture uses a bidirectional LSTM layer. The first layer reshapes the image data to have the correct dimensions for the LSTM layer. The implemented LSTM model architecture is presented in Table 2. In total, this model has 3,76,955 trainable parameters.

5. Recurrent Neural Network — Gated Recurrent Unit

The Gated Recurrent Unit model architecture is similar to the presented LSTM model architecture but with the LSTM layer switched to the GRU layer and different

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 2. Long short-term memory — model architecture.

	Type	Kernel	Kernel size	Notes	Input shape
1	Com2D	8	7×7	Activation = relu	$224 \times 224 \times 3$
2	Max Poolmg2D		2×2		$224 \times 224 \times 8$
3	BatchNormalization				$112 \times 112 \times 8$
4	Com 2D	16	5×5	Activation = relu	$112 \times 112 \times 8$
5	MaxPoolmg2D		2×2		$112 \times 112 \times 16$
6	BatchNormalization				$56 \times 56 \times 16$
7	Com2D	32	3×3	Activation = relu	$56 \times 56 \times 16$
8	Max Pool mg2D		2×2		$56 \times 56 \times 32$
9	BatchNormalization				$28 \times 28 \times 32$
10	Com2D	64	3×3	Activation = relu	$28 \times 28 \times 32$
11	MaxPoolmg2D		2×2		$28 \times 28 \times 64$
12	BatchNormalization				$14 \times 14 \times 64$
13	Com2D	128	3×3	Activation = relu	$14 \times 14 \times 64$
14	Max Pool mg2D		2×2		$14 \times 14 \times 128$
15	BatchNormalization				$7 \times 7 \times 128$
16	Conv2D	256	3×3	Activation =relu	$7 \times 7 \times 128$
17	Dropout			Rate = 0.5	$7 \times 7 \times 256$
18	MaxPoolmg2D		2×2		$7 \times 7 \times 256$
19	Flatten				$4 \times 4 \times 256$
20	Dropout			Rate = 0.5	4096
21	Dense	128		Activation = relu	4096
22	Dense	91		Activation = softmax	128

Table 3. Gated recurrent unit — model architecture.

	Type	Kernel	Notes	Input shape
1	Reshape		TimeDistributed, Target = -1	$224 \times 224 \times 3$
2	Dense	64	TimeDistributed; Activation = tanh	224×672
3	GRU	128	BiDirectional	224×64
4	Dense	128	Activation = relu	224×256
5	MaxPoolinglD			224×128
6	Dense	64	Activation = relu	112×128
7	Flatten			112×64
8	Dropout		Rate = 0.5	7168
9	Dense	32	Activation = relu	7168
10	Dense	91	Activation = softmax	32

units on the dense layers after the MaxPooling1D layer. Table 3 presents the implemented Gated Recurrent Unit model architecture. In total, this model has 4,65,627 trainable parameters.

6. Convolutional Recurrent Neural Network — Long Short-Term Memory

The Convolutional Recurrent Neural Network (CRNN) using LSTM model architecture is based on merging CNN and LSTM model architectures. Layers have been

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 4. CRNN–LSTM - Model architecture.

	Type	Kernel	Notes	Input shape
1	Conv2D	32	Activation = relu	$224 \times 224 \times 3$
2	MaxPooling2D			$224 \times 224 \times 32$
3	Conv2D	64	Activation = relu	$112 \times 112 \times 32$
4	MaxPooling2D			$112 \times 112 \times 64$
5	Conv2D	128	Activation = relu	$56 \times 56 \times 64$
6	MaxPooling2D			$56 \times 56 \times 128$
7	Conv2D	256	Activation = relu	$28 \times 28 \times 128$
8	MaxPooling2D			$28 \times 28 \times 128$
9	Dropout		Rate = 0.5	$14 \times 14 \times 256$
10	Reshape		TimeDistributed; Target = -1	$14 \times 14 \times 256$
11	LSTM	256	BiDirectional	14×3584
12	MaxPoolinglD			7×512
13	Dropout		Rate = 0.5	7×512
14	LSTM	128	Bi Directional	7×512
15	MaxPoolinglD			7×256
16	Flatten			3×256
17	Dropout		Rate = 0.5	768
18	Dense	32	Activation = relu	768
19	Dense	91	Activation = softmax	64

added, removed, and modified in order to get better results. A dropout layer is added between the Convolutional layers and the LSTM layers in order to reduce overfitting. In Table 4, we present the implemented CRNN–LSTM model architecture. In total, this model has 8,981,307 trainable parameters.

7. Convolutional Recurrent Neural Network — Gated Recurrent Unit

The Convolutional Recurrent Neural Network — Gated Recurrent Unit (CRNN–GRU) model architecture is based on the presented CRNN–LSTM but with the LSTM layer switched to the GRU layer, with double the amount of kernel units. The second to last dense layer also has double the amount of kernel units. In Table 5, we

Table 5. CRNN–LSTM — model architecture.

	Type	Kernel	Notes	Input shape
1	Conv2D	32	Activation = relu	$224 \times 224 \times 3$
2	MaxPooling2D			$224 \times 224 \times 32$
3	Conv2D	64	Activation = relu	$112 \times 112 \times 32$
4	MaxPooling2D			$112 \times 112 \times 64$
5	Conv2D	128	Activation = relu	$56 \times 56 \times 64$
6	MaxPooling2D			$56 \times 56 \times 128$
7	Conv2D	256	Activation = relu	$28 \times 28 \times 128$
8	MaxPoo!ing2D			$28 \times 28 \times 256$
9	Dropout		Rate = 0.5	$14 \times 14 \times 256$
10	Reshape		TimeDistributed; Target = -1	$14 \times 14 \times 256$

Table 5. (*Continued*)

	Type	Kernel	Notes	Input shape
11	GRU	512	BiDirectional	14×3584
12	MaxPoolinglD			14×1024
13	Dropout		Rate = 0.5	7×1024
14	GRU	256	BiDirectional	7×1024
15	MaxPoolinglD			7×512
16	Flatten			3×512
17	Dropout		Rate = 0.5	1536
18	Dense	64	Activation = relu	1536
19	Dense	91	Activation = softmax	64

present the implemented CRNN–GRU model architecture. In total, this model has 150,65,915 trainable parameters.

7.1. *Implementation*

Our solution, implemented in Python with TensorFlow framework, is presented in Fig. 8. This solution has two independent parts: Data pre-processing and deep learning.

The data pre-processing part is composed by:

- dataset_manager.py, responsible for the dataset operations.
- audio_preprocessor.py, responsible for the loading of the audio files, the pre-processing and the feature extraction, and exports the features.
- dataset.py contains three classes: Dataset, Class (contains a list of Samples and the name of the Bird Species), and Sample (contains the original file name and the sample number, the audio time series, the Mel Frequency Cepstrum Coefficients, and the Mel Spectrogram extracted features).
- mp3_to_wav.py and normalize_audio.py to convert from MP3 to WAV and normalize the audio files.

The deep learning part is composed by:

- deep_learning.py containing
  - DeepLearning class, providing the methods:
    - generate_data(): responsible to the training and validation sets, using the ImageDataGenerator from the back-end Keras of the TensorFlow framework.
    - train(): fits the model with the training set and validates with the validation set. The method gets the model from the models.py Python file.
  - compare_models() and compare_features() to automatically execute different experiment methodologies.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

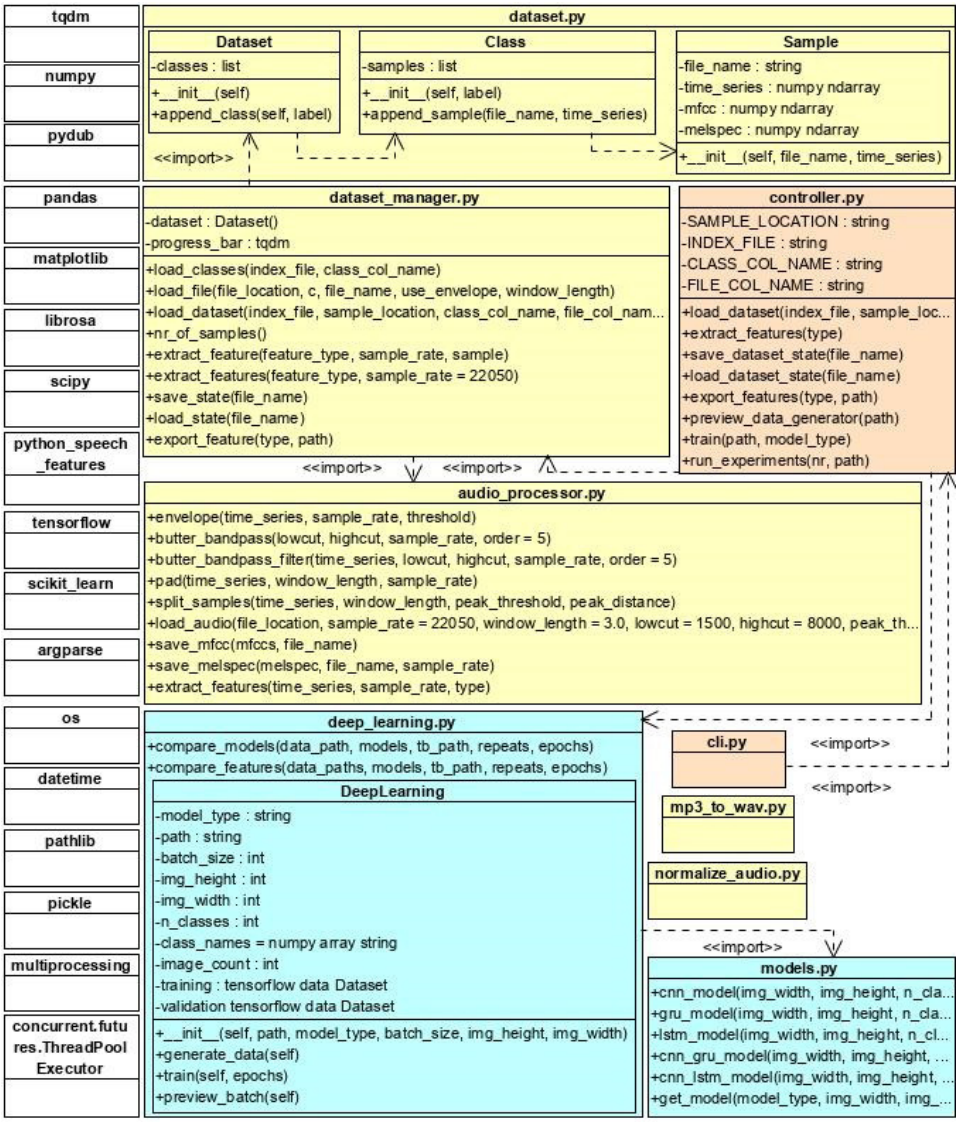

Fig. 8. Class and library diagram.

8. Evaluation

Multiple combinations of feature extraction methods and deep learning models were tested. In total, there are 10 combinations to be tested, five deep learning models and two feature extraction methods: CNN with MFCC and Mel Spectrogram; RNN LSTM with MFCC and Mel Spectrogram; RNN GRU with MFCC and Mel Spectrogram; CRNN LSTM with MFCC and Mel Spectrogram, and CRNN GRU with MFCC and Mel Spectrogram.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

8.1. *Evaluation metrics*

The methodology evaluation in the experiments was the holdout. The data were randomly split into two folds, the training set (80% of the data), and the validation set (20% of the data). The main metric used to compare the deep learning models was the average recall (Eq. (2)) per bird species and accuracy (Eq. (1)).

$$\text{Accuracy} = \frac{\text{CorrectPredictions}}{\text{TotalPredictions}}, \tag{1}$$

$$\text{Recall} = \frac{\text{TruePositive}}{\text{TruePositive} + \text{FalseNegative}} \tag{2}$$

8.2. *Experiments*

We evaluated 10 combinations: Two feature extraction methods and five deep learning models. These combinations are as follows:

- CNN with: MFCC and Mel Spectrogram,
- RNN LSTM with: MFCC and Mel Spectrogram,
- RNN GRU with: MFCC and Mel Spectrogram,
- CRNN LSTM with: MFCC and Mel Spectrogram,
- CRNN GRU with: MFCC and Mel Spectrogram.

The experiments were conducted in the following environment:

- OS: Ubuntu 20.04 LTS (Focal Fossa),
- Python: version 3.6,
- TensorFlow: version 2.2,
- Processor: AMD Ryzen 9 3900X,
- RAM: 32 GB,
- Graphics Card: AMD RX 580X 8 GB (RadeonOpenCompute 3.5.0).

8.3. *Best results*

In Table 6, we can see the summary of the results for the proposed models.

From the experiments, the feature extraction Mel Spectrogram with 3 s sample lengths obtained better results than MFCCs. The best model architecture is the CRNN using Gated Recurrent Unit layers, as it achieved the best results when compared to the other implementations in terms of recall and accuracy.

Our best methodology obtained in this work (Mel Spectrogram with 3 sec length and Convolutional Recurrent Neural Network using Gated Recurrent Units) was also compared with a previous approach by Hiatt[22] for the same dataset. We have used the same holdout split (33% for test set). However, we do not have specific information about which examples were used for training and testing by Hiatt. The results obtained are presented in Table 7. Although the results were obtained using

Table 6. Extracted features and deep learning model architecture combination comparison.

Comb		At maximum recall (Validation set)							
		Validation set					Training set		
F	Models	Recall	Acc.	Loss	Epoch	Time	Recall	Acc.	Loss
MFCCs 3s	CNN	40.50%	45.41%	3.040	47	57m	98.82%	99.13%	0.0787
	LSTM	32.44%	38.01%	3.778	47	91m	77.86%	84.65%	0.6491
	GRU	35.50%	39.53%	3.979	46	107m	85.69%	89.77%	0.4622
	CRNN LSTM	**44.76%**	46.10%	4.233	42	74m	96.17%	97.01%	0.1556
	CRNN GRU	44.40%	47.81%	3.470	41	88m	90.80%	92.79%	0.3498
MFCCs 1.5s	CNN	38.82%	44.44%	3.134	45	52m	98.19%	98.76%	0.1116
	LSTM	27.27%	33.65%	3.909	45	89m	71.99%	80.79%	0.7931
	GRU	31.36%	36.23%	4.097	46	111m	79.56%	85.26%	0.6354
	CRNN LSTM	42.36%	43.95%	4.404	43	76m	91.97%	93.81%	0.2796
	CRNN GRU	**43.35%**	46.47%	3.539	40	82m	90.82%	92.64%	0.3476
Mel Spectrogram 3s	CNN	47.51%	43.05%	2.574	40	68m	99.05%	99.37%	0.0761
	LSTM	26.27%	33.32%	3.832	44	95m	65.85%	75.80%	0.9608
	GRU	28.23%	33.48%	4.271	43	110m	75.96%	83.36%	0.7034
	CRNN LSTM	47.95%	50.30%	3.475	48	93m	94.40%	95.60%	0.2140
	CRNN GRU	**50.17%**	53.13%	3.003	45	98m	90.64%	92.70%	0.3659
Mel Spectrogram 1.5s	CNN	44.21%	49.28%	2.957	41	68m	98.76%	99.14%	0.0813
	LSTM	24.62%	32.51%	3.790	47	101m	62.68%	73.29%	1.051
	GRU	27.56%	32.51%	4.071	46	115m	79.56%	85.46%	0.6354
	CRNN LSTM	46.46%	48.48%	3.763	40	78m	91.30%	93.39%	0.3109
	CRNN GRU	**47.92%**	50.71%	3.286	46	102m	89.74%	91.95%	0.3935

different splits, we can observe that our methodology performed considerably better, achieving an accuracy of 44.26% on the validation set, while Hiatt's obtained 19.27%. This large difference allows us to believe in the superiority of our approach on this dataset.

Table 7. Results comparison.

	At minimum loss (Validation set)			
	Validation set			Training set
Implementation	Accuracy	Loss	Epoch	Accuracy
This Work	44.26%	2.657	6	70.74%
Hiatt 2019	19.27%	3.605	85	20.44%

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

9. Conclusions

The goal of this paper is to contribute to the development of a reliable methodology that classifies bird species by their chirp, using deep learning techniques. We have presented an audio pre-processing methodology that normalized the audio, removed noise outside of the bird vocalization range, and split an audio file into multiple equal length samples by detecting syllables or peaks. The audio samples then have their features extracted in one of the two methods. These are the Mel Spectrograms and the Mel Frequency Cepstrum Coefficients. The classification methodology for this solution uses one of the two types of extracted features to train a Deep Learning model for predicting the species of a bird, by first pre-processing the audio file using the same audio pre-processing methodology that was used on the dataset used to train the model. An optimization methodology was created with the intent of testing and refining both the pre-processing methodology and the deep learning models. During the experiments conducted using this methodology, multiple alternative deep learning model architectures were tested with alternatives of CNNs, RNNs, and CRNNs. The best results were obtained with the CRNN using Gated Recurrent Unit layers, the CRNN using LSTM layers, and the CNN. The final experiment is comparing the best methodology obtained in this work with another implementation,[22] while using the same experiment environment. Our methodology performed better, achieving an accuracy of 44.26% on the validation set, while Hiatt's obtained 19.27%.

In the future, we intend to improve the audio pre-processing methodology with other audio pre-processing and feature extraction techniques such as data preparation and data augmentation techniques and other feature extraction methods. We also intend to use further optimized deep learning models with different architectures and hyperparameters. Additionally, we will work on the discrepancy between validation and training results to reduce the possibility of overfitting. Transfer learning could also be used.[30] Furthermore, we intend to conduct more experiments using different types of bird sound datasets.

Acknowledgments

This work is financed by National Funds through the Portuguese funding agency, FCT — Fundação para a Ciência e a Tecnologia, within project UIDB/50014/2020. This paper is an extended version of the ACIIDS 2021 paper "Automatic Identification of Bird Species from Audio".

References

1. Cağan H Sekercioğlu, Richard B. Primack and Janice Wormworth. The effects of climate change on tropical birds. *Biological Conservation* **148**(1) (2012) 1–18. 10.1016/j.biocon.2011.10.019.

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

2. C. A. Morrison, A. Auniš, Z. Benkő *et al.*, Bird population declines and species turnover are changing the acoustic properties of spring soundscapes, *Nat. Commun.* **12** (2021) 6217, https://doi.org/10.1038/s41467-021-26488-1.
3. P. Gavali, Ms. Prachi Abhijeet Mhetre , Ms. Neha Chandrakhant Patil , Ms. Nikita Suresh Bamane, Ms. Harshal Dipak Buva, Bird species identification using deep learning, *Int. J. Eng. Res. Technol.* **8**(4) (2019), https://www.ijert.org/bird-species-identification-using-deep-learning.
4. V. Boddapati *et al.*, Classifying environmental sounds using image recognition networks, *Procedia Comput. Sci.* **112** (2017) 2048–2056, doi: 10.1016/j.procs.2017.08.250.
5. C.-J. Huang *et al.*, Frog classification using machine learning techniques, *Expert Syst. Appl.* **36**(2) (2019) 3737–3743, doi: 10.1016/j.eswa.2008.02.059.
6. J. Colonna *et al.*, Automatic classification of anuran sounds using convolutional neural networks, in *Proc. Int. Conf. Computer Science and Software Engineering*, pp. 73–78, doi: 10.1145/2948992.2949016.
7. S. Fagerlund, Bird species recognition using support vector machines, *EURASIP J. Adv. Signal Process.* **2007** (2007) 1–8, doi: 10.1155/2007/38637.
8. R. Wielgat *et al.*, HFCC based recognition of bird species, *Signal Processing Algorithms, Architectures, Arrangements, and Applications SPA 2007*, 7 September 2007, Poznan, Poland, pp. 129–134, doi: 10.1109/SPA.2007.5903313.
9. L. Roberts, Understanding the Mel Spectrogram - Analytics Vidhya - Medium, *Medium*, https://medium.com/analytics-vidhya/understanding-the-melspectrogram-fca2a-fa2ce53.
10. S. Davis and P. Mermelstein, Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences, *IEEE Trans. Acoust. Speech Signal Process.* **28**(4) (1980) 357–366, doi: 10.1109/TASSP.1980.1163420.
11. M. Kortas, Sound-Based Bird Classification, *Medium*, https://towardsdatascience.com/sound-based-bird-classification-965d0ecacb2b, https://www.deeplearningbook.org/contents/intro.htmls, last accessed 14 November 2020.
12. Y. LeCun, Y. Bengio and G. Hinton, Deep learning. *Nature* **521** (2015) 436–444, https://doi.org/10.1038/nature14539.
13. C. Nicholson, A Beginner's Guide to LSTMs and Recurrent Neural Networks, https://pathmind.com/wiki/lstm, last accessed 27 December 2019.
14. M. Nguyen, Illustrated guide to LSTM's and GRU's: A step by step explanation, in Medium, https://towardsdatascience.com/illustrated-guideto-lstms-and- gru- s- a- step-by- step- explanation- 44e9eb85bf21, last accessed 28 December2019.
15. Hochreiter and J. Schmidhuber, Long short-term memory, *Neural Comput.* **9**(8) (1997) 1735–1780, doi: 10.1162/neco.1997.9.8.1735.
16. C. Olah, Understanding LSTM Networks, https://colah.github.
17. S. Kostadinov, Understanding GRU Networks, *Medium*, https:// towardsdatascience.com/understanding-gru-networks-2ef37df6c9be, last accessed 28 December 2019.
18. K. Choi *et al.*, Convolutional Recurrent Neural Networks for Music Classification, arXiv: 1609.04243 [cs.NE] (2016).
19. M. Lasseck, Bird Species identification in soundscapes, *CLEF*, 9–12 September 2019, Lugano, Switzerland.
20. S. Kahl *et al.*, Overview of BirdCLEF 2019: Large-scale bird recognition in soundscapes, CLEF,, 9–12 September 2019, Lugano, Switzerland.
21. S. Hiatt, Avian Vocalizations - Report, Kaggle, https://www.kaggle.com/samhiatt/avian-vocalizations-report.
22. S. Butterworth *et al.*, On the theory of filter amplifiers, *Wireless Eng.* **7**(6) (1930) 536–541.

23. J. Lyons, jameslyons/python_speech_features: release v0.6.1, Zenodo, doi:10.5281/zenodo.3607820, https://odr.chalmers.se/handle/20.500.12380/249467 (2020).
24. B. McFee *et al.*, librosa/librosa: 0.7.2, doi: 10.5281/zenodo.3606573 (2020).
25. J. D. Hunter, Matplotlib: A 2D graphics environment, *Comput. Sci. Eng.* **9**(3) (2007) 90–95, doi: 10.1109/MCSE.2007.55.
26. M. Abadi *et al.*, TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Software available from tensorflow.org, https://www.tensorflow.org/ (2015).
27. F. Chollet *et al.*, Keras, https://keras.io (2015).
28. S. Adams, Audio-Classification, https://github.com/seth814/AudioClassiftion/tree/2f0032d81dcfa3d662cab1c1c4e7e30520f7edd6, last accessed 7 June 2020.
29. F. Doukkali, Batch normalization in neural networks — towards data science, in Medium, (2019) https://towardsdatascience.com/batch-normalization-in-neural-networks-1ac91516821c.
30. J. Xie, C. Ding, W. Li, and C. Cai, Audio-only bird species automated identification method with limited training data based on multi-channel deep convolutional neural networks, arXiv:abs/1803.01107 (2018).

Vietnam J. Comp. Sci. 2023.10:39-54. Downloaded from www.worldscientific.com
by 178.52.222.17 on 11/27/25. Re-use and distribution is strictly not permitted, except for Open Access articles.